Subtraction Gates: Another Way to Learn Long-Term Dependencies in Recurrent Neural Networks

Tao He, *Student Member, IEEE*, Hua Mao, *Member, IEEE*, and Zhang Yi, *Fellow, IEEE*

***Abstract*— Recurrent neural networks (RNNs) can remember temporal contextual information over various time steps. The well-known gradient vanishing/explosion problem restricts the ability of RNNs to learn long-term dependencies. The gate mechanism is a well-developed method for learning long-term dependencies in long short-term memory (LSTM) models and their variants. These models usually take the multiplication terms as gates to control the input and output of RNNs during forwarding computation and to ensure a constant error flow during training. In this article, we propose the use of *subtraction* terms as another type of gates to learn long-term dependencies. Specifically, the multiplication gates are replaced by *subtraction* gates, and the activations of RNNs input and output are directly controlled by subtracting the subtrahend terms. The error flows remain constant, as the linear identity connection is retained during training. The proposed *subtraction* gates have more flexible options of internal activation functions than the multiplication gates of LSTM. The experimental results using the proposed Subtraction RNN (SRNN) indicate comparable performances to LSTM and gated recurrent unit in the Embedded Reber Grammar, Penn Tree Bank, and Pixel-by-Pixel MNIST experiments. To achieve these results, the SRNN requires approximate three-quarters of the parameters used by LSTM. We also show that a hybrid model combining multiplication forget gates and *subtraction* gates could achieve good performance.**

***Index Terms*— Gates, long short-term memory (LSTM), recurrent neural networks (RNNs), subtraction.**

I. Introduction

RECURRENT neural networks (RNNs) have been widely used for sequence learning tasks, such as speech recognition [1], machine translation [2], and handwriting recognition [3]. Many variants of normal RNNs have been proposed in recent years, such as Lotka–Volterra RNNs [4]–[6], delayed RNNs [7], [8], oscillator RNNs [9], and gated RNNs [10], [11]. In theory, RNNs handle time-varying sequence learning tasks by storing a representation of the inputs over random time lags. However, the training of RNNs is usually based on gradient descent algorithms, such as real-time recurrent learning (RTRL) or backpropagation through time (BPTT) [12], which can result in the well-known gradient vanishing/explosion problem. Fortunately, convergence analysis has enabled the performance of RNNs to be extensively studied [13]–[15]. In this article, we attempt to enhance the ability of RNNs to retain long memory.

The gradient vanishing/explosion problem is the main reason why RNNs have difficulty learning long-term dependencies. The breakthrough in solving this problem was long short-term memory (LSTM) [10], which introduced a gate mechanism and constant error carrousel (CEC), which is a linear self-connection of recurrent neurons. By adding a linear identity function $f(x) = x$, LSTM can retain a constant error flow during training. However, the introduction of CEC invokes the more pronounced input and output conflict problem. Thus, input and output gates were used to control how much information should be input to and output from the network, respectively [10]. Moreover, forget gates [16] controlled the values kept in the memory cells of LSTM, allowing LSTM to deal with continual learning tasks. These gates provide a more flexible way to control the error flow when handling the gradient vanishing/explosion problem.

Note that the gates of LSTM are based on the multiplication terms. In this article, we define the gates with multiplication terms as multiplication gates. However, are multiplication gates the only type of gate capable of learning long-term dependencies? The concept of gates is widely used in many domains. For example, variable resistors, which are used to control the current in electronic circuits, typically operate as a gate. By adjusting the value of the variable resistor, we control the quantity of electricity that can pass through. In water conservation projects, dams are often built as a big gate to control the flow of water in rivers. However, using a dam is not the only way to control the water flow. Another construction is a water-diversion levee, which is built to redirect a portion of the river's flow.

Controlling the message flow in RNNs is similar to controlling the water flow in a river. Motivated by this novel idea, we propose a close analogy by directly throwing away unnecessary portions of activations using a simple subtraction gate. Different from multiplication gates: $f \odot \sigma$ in Fig. 1(a), the formulation of subtraction gates is $f - \sigma$, as shown in Fig. 1(b), where $\odot$ indicates elementwise multiplication. f and σ are the activation functions of the recurrent neurons and gates, respectively. Among them, σ directly learns the information dropped from f. We refer to RNNs with subtraction gates

Manuscript received July 22, 2019; revised February 1, 2020 and August 4, 2020; accepted December 7, 2020. Date of publication December 29, 2020; date of current version April 5, 2022. This work was supported in part by the National Major Science and Technology Projects of China under Grant 2018AAA0100201 and in part by the National Natural Science of China under Grant 61836011. *(Corresponding author: Zhang Yi.)*

The authors are with the College of Computer Science, Sichuan University, Chengdu 610065, China (e-mail: zhangyi@scu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TNNLS.2020.3043752.

Digital Object Identifier 10.1109/TNNLS.2020.3043752

2162-237X © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

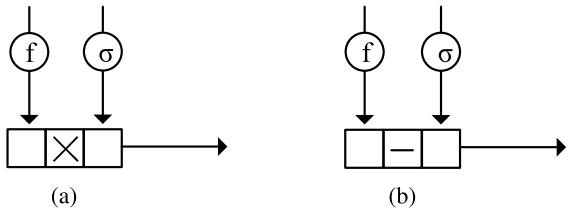


Fig. 1. Illustration of multiplication and subtraction gates. (a) $f \times \sigma$. (b) $f - \sigma$.

as subtraction recurrent neural networks (SRNNs). The main contributions of this article can be summarized as follows.

1) We propose a novel subtraction gate-based RNN model. By replacing the multiplication term with the proposed subtraction term, the SRNN directly controls how much information should flow into and be dropped from the network. The SRNN retains the linear identity connection to ensure a constant error flow during training.
2) It is proved that the proposed SRNN can learn long-term memories through the truncated RTRL algorithm. We verify that the SRNN still works with the full gradient BPTT algorithm in our experiments and investigates that adding and subtracting a bias from the input pre-activation neurons helps with the training process.
3) An SRNN with the same number of recurrent neurons as LSTM requires approximate three-quarters as many parameters because the subtraction gate applied in the memory cells involves no parameters. A hybrid RNN model combining multiplication forget gates and subtraction gates, called F-SRNN, also exhibits good performance in our experiments.

In the remainder of this article, we first introduce related works on dealing with the gradient vanishing/explosion problem in Section II. The preliminaries are introduced in Section III. In Section IV, we first detailed introduce the proposed SRNN. Then, the underlying theory behind the subtraction gates is explained. Finally, we present a real-time learning algorithm based on RTRL to train the SRNN. In Section V, the proposed SRNN is validated by the adding problem, conducting Embedded Reber Grammar (ERG), PTB language modeling, Pixel-by-Pixel MNIST, and Yelp2013 document classification experiments. Finally, we conclude this article in Section VI.

II. Related Work

To evaluate the benefit of LSTM's gates, eight variants of LSTM were compared across tasks, such as speech recognition, handwriting recognition, and polyphonic music modeling in [17]. It was concluded that gates (especially forget gates) are the most critical component of LSTM. Recently, some studies have exploited the gate mechanism of LSTM. For instance, gated recurrent unit (GRU) [11] was proposed to use coupled forget gates and input gates, which were called update gates. It has been found that GRUs are comparable to LSTM for polyphonic music and speech signal modeling [18]. Peephole connections, proposed in [19], could learn the fine distinction between sequences of spikes separated by either 50 or 49 discrete time steps. An LSTM with peephole connections could learn to generate stable sequences of highly nonlinear, precisely timed spikes. Auxiliary memory units [20] (AMU) was a typical application of gates. AMU provided an auxiliary memory neuron to maintain memory and separated the memory and output in a recurrent unit to solve the gradient vanishing/explosion problem. Grid LSTM [21] further expanded the potential of LSTM. It provided a unified way of using LSTM for both deep and sequential computation.

The gradient vanishing/explosion problem also happened in feed-forward neural networks (FNNs) and restrained the training of very deep FNNs. Therefore, in [22], multiplication gates were applied in highway networks, which allow unimpeded information flow across many layers. The main contribution of highway networks is to use adaptive gating units to regulate the information flow. The recurrent highway network [23], which is a variant of the highway network, extended RNNs to allow step-to-step transition depths larger than one. The multiplication gates were not the only way to deal with the gradient vanishing/explosion problem in FNNs. In [24], the proposed residual networks (ResNets) did not apply multiplication gates but directly applied shortcut connections in networks. The experiment results showed that a ResNet of 1000 layers trained on CIFAR-10 achieved high performance. Similarly, dense convolutional network [25] even connected each layer to every other layer in a feed-forward fashion.

III. Preliminaries

This section first reviews the gradient vanishing/explosion problem. We then explain how the linear self-connection of recurrent neurons and the gates can solve this problem. The statement of notations are as follows: bold capitals denote a matrix, such as $\mathbf{M}$; bold lowercases denote a vector, such as $\mathbf{v}$; and italic characters with subscripts denote a variable, such as x_i. Especially, we use italic capitals to denote pre-activation, such as Z_i.

A. Gradient Vanishing/Explosion Problem

Let the input and output of an RNN be $\mathbf{x}(t)$ and $\mathbf{y}(t)$ at time t. The forward computation of RNNs can be formulated as

$$\begin{cases} Z_i(t+1) = \sum_j w_{ij} y_j(t) + \sum_k w_{ik} x_k(t) \\ y_i(t+1) = f(Z_i(t+1)) \end{cases} \tag{1}$$

where f denotes the activation function, w_{ij} denotes the connections from the recurrent neuron j to i, and w_{ik} denotes the connections from input neuron k to recurrent neuron i. Equation (1) denotes the dynamics of a standard RNN. Given the total loss for an input sequence

$$\mathcal{J}_{\text{total}} = \sum_{t \geqslant 1} \mathcal{J}(t) \tag{2}$$

the training of the RNN involves updating w_{ij} by computing the total loss as

$$\frac{\partial \mathcal{J}_{\text{total}}}{\partial w_{ij}} = \sum_{t \geqslant 1} \frac{\partial \mathcal{J}(t)}{\partial w_{ij}}. \tag{3}$$

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

Using the chain rule, the partial derivative of $\mathcal{J}_{\text{total}}$ with respect to w_{ij} is

$$\frac{\partial \mathcal{J}_{\text{total}}}{\partial w_{ij}} = \sum_{t \geqslant 1} \frac{\partial \mathcal{J}(t)}{\partial \mathbf{y}(t)} \cdot \frac{\partial \mathbf{y}(t)}{\partial w_{ij}} = \sum_{t \geqslant 1} \sum_{1 \leqslant \tau \leqslant t} \frac{\partial \mathcal{J}(t)}{\partial \mathbf{y}(t)} \cdot \frac{\partial \mathbf{y}(t)}{\partial \mathbf{y}(\tau)} \cdot \frac{\partial \mathbf{y}^{+}(\tau)}{\partial w_{ij}} \tag{4}$$

where $\mathbf{y}(t)$ and $\mathbf{y}(\tau) \in \mathbb{R}^n$, are the output vectors at time t and τ. $((\partial \mathbf{y}^{+}(\tau))/(\partial w_{ij}))$ indicates the immediate partial derivative with respect to w_{ij} at time τ [26]. This is calculated as

$$\frac{\partial y_k^{+}(\tau)}{\partial w_{ij}} = u_{ki} \cdot f'(Z_k(\tau)) \cdot y_j(\tau - 1) \tag{5}$$

where u_{ki} is the Kronecker function: if $k == i$, $u_{ki} = 1$; otherwise, $u_{ki} = 0$.

The partial derivative $((\partial \mathbf{y}(t))/(\partial \mathbf{y}(\tau)))$ can be expanded as

$$\frac{\partial \mathbf{y}(t)}{\partial \mathbf{y}(\tau)} = \prod_{\tau < \epsilon \leqslant t} \frac{\partial \mathbf{y}(\epsilon)}{\partial \mathbf{y}(\epsilon - 1)} = \prod_{\tau < \epsilon \leqslant t} \mathbf{W}_{\text{rec}}^T \cdot \text{diag}\big[f'\big(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)\big)\big] \tag{6}$$

where $\mathbf{W}_{\text{rec}}^T \in \mathbb{R}^{n \times n}$ is the transposition of the recurrent weight matrix and diag$[\cdot]$ indicates the conversion of a vector into a diagonal matrix. In the gradient explosion problem, the norm of the term $((\partial \mathbf{y}(t))/(\partial \mathbf{y}(\tau)))$ grows exponentially as t increases. In the gradient vanishing problem, the norm of the term $((\partial \mathbf{y}(t))/(\partial \mathbf{y}(\tau)))$ decreases rapidly to norm 0. The conditions for gradient vanishing and explosion have been identified in [23] and [26]. Let γ be the maximal bound on $f'(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)))$ and ξ be the largest singular value of $\mathbf{W}_{\text{rec}}^T$. Then, the norm of the Jacobian satisfies

$$\left\| \frac{\partial \mathbf{y}(\epsilon)}{\partial \mathbf{y}(\epsilon - 1)} \right\| \leq \left\| \mathbf{W}_{\text{rec}}^T \right\| \cdot \left\| \text{diag}\big[f'\big(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)\big)\big] \right\| \leq \xi \cdot \gamma. \tag{7}$$

The sufficient condition for gradient vanishing is $\xi \cdot \gamma < 1$ for all ϵ. Similarly, the necessary condition for exploding gradients is $\xi \cdot \gamma > 1$. Because of the product of (6), the gradient vanishing/explosion problem can readily appear.

B. Self-Connections of Recurrent Neurons

One novel intuition of LSTM is to involve the linear self-connections [10]. Let us focus on the connection from neuron j to neuron i. By adding a self-connected identity, the formulation of the RNN becomes

$$y_i(t + 1) = y_i(t) + f(Z_i(t + 1)). \tag{8}$$

By introducing the self-connected identity, the Jacobian $((\partial \mathbf{y}(\epsilon))/(\partial \mathbf{y}(\epsilon - 1)))$ can be calculated as

$$\frac{\partial \mathbf{y}(\epsilon)}{\partial \mathbf{y}(\epsilon - 1)} = \mathbf{I} + \mathbf{W}_{\text{rec}}^T \cdot \text{diag}\big[f'\big(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)\big)\big] \tag{9}$$

where $\mathbf{I} \in \mathbb{R}^{n \times n}$ is the identity matrix. This ensures a constant error flow and provides a way of avoiding the gradient vanishing/explosion problem by adjusting the norm of $\mathbf{W}_{\text{rec}}^T \cdot \text{diag}[f'(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)))]$ to be close to 0. More precisely, the essential problem of maintaining a constant gradient is equivalent to keeping $((\partial \mathbf{y}(\epsilon))/(\partial \mathbf{y}(\epsilon - 1)))$ as a constant orthogonal matrix. However, the difficulty is to ensure the Jacobian remains orthogonal after many weight updates. Recently, many studies have developed orthogonal matrix methods [27], [28]. As the identity matrix $\mathbf{I}$ is orthogonal, the self-connections attempt to obtain a constant error flow.

The RNN with this self-connected identity forms the CEC structure [10]. The linear self-connections introduce a new problem because of the constant summarization of memory cells. When a sequence is input to an RNN with CEC, the nonlinear activations $f(Z_i(t + 1))$ are constantly added to the output $y_i(t + 1)$ as t increases. Thus, the output of the RNN with CEC usually explodes, especially when the input sequence is long. It remains a difficult problem to keep the gradient term $\mathbf{W}_{\text{rec}}^T \cdot \text{diag}[f'(\mathbf{W}_{\text{rec}}^T \mathbf{y}(\epsilon - 1)))]$ close to 0.

C. Multiplication Gates

The multiplication gates of LSTM control how much information should be input to, retained by, and output from the memory cells. The multiplication gate mechanism can be formulated as $f \odot \sigma$, where $\odot$ indicates the element-wise multiplication. The corresponding input gates are formulated as follows:

$$\begin{cases} I_i(t + 1) = \sum_j w_{ij}^I y_j(t) + \sum_k w_{ik}^I x_k(t) \\ s_i(t + 1) = s_i(t) + g(Z_i(t + 1)) \odot \sigma_I(I_i(t + 1)) \end{cases} \tag{10}$$

where σ_I indicates the input gate function, and s and g indicate the internal memory state and the input activation function, respectively. To control information output from the memory cells, LSTM uses a nonlinear function h to compress the output and then applies the multiplication gate mechanism, which can be formulated as

$$\begin{cases} O_i(t + 1) = \sum_j w_{ij}^O y_j(t) + \sum_k w_{ik}^O x_k(t) \\ y_i(t + 1) = h(s_i(t + 1)) \odot \sigma_O(O_i(t + 1)) \end{cases} \tag{11}$$

where σ_O indicates the output gate function and $y_i(t + 1)$ indicates the output of LSTM. In LSTM, g and h are usually hyperbolic tangent functions and σ is the logistic sigmoid function. Thus, the input gates control how much information is memorized and summarized in $s_i(t + 1)$, and the output gates control how much information is read from the memory cells.

Despite the use of input and output gates, LSTM cannot process the continual sequence learning problem. When a continual sequence is input, the memory cell is not reset and the network eventually breaks down. To overcome this problem, forget gates are applied in linear self-connections [16]. The formulation of forget gates is

$$\begin{cases} F_i(t + 1) = \sum_j w_{ij}^F y_j(t) + \sum_k w_{ik}^F x_k(t) \\ s_i(t + 1) = s_i(t) \odot \sigma_F(F_i(t + 1)) + g(Z_i(t + 1)). \end{cases} \tag{12}$$

In theory, the forget gates destroy the constant error flow of linear CEC during the training stage. Thus, before training the

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

LSTM with forget gates, the bias weights of the forget gates are usually initialized with positive values to keep them fully opened at the start of the training phase. This technique was recently rediscovered [18], and it was found that adding a bias of 1 to the forget gate is a very important step when training LSTM.

IV. Methods

A. Proposed Subtraction Recurrent Neural Networks

The gates developed in previous studies are multiplication terms that can be formulated as $f \odot \sigma$, where $\odot$ indicates elementwise multiplication. f and σ are the activation functions of the recurrent neurons and gates, respectively. Typically, σ is a logistic sigmoid function with activation values between 0 and 1. The elementwise multiplication $\odot$ acts like a gate and σ can be thought of as the key to the gate. If $\sigma(\cdot) = 0$, the gate is fully closed, whereas if $\sigma(\cdot) = 1$, the gate is fully open.

The formulation of $f \odot \sigma$ is called a gate because of the product term. Intuitively, the direct way to control the information in RNNs is to subtract a portion from the input, output, and memory cells. From this perspective, the gate $f \odot \sigma$ could be rewritten as $f - f \odot (1-\sigma)$. For simplification, we let $\hat{\sigma} = f \odot (1-\sigma)$. The gate can then be viewed as $f - \hat{\sigma}$, where $\hat{\sigma} \in (0, f)$ because σ is a fixed logistic sigmoid function. The control mechanism can be explained as follows.

1) If $\hat{\sigma}(\cdot) = f(\cdot)$, no information is transferred to the next state of the RNN.
2) If $\hat{\sigma}(\cdot) = 0$, all information is transferred to the next state of the RNN.

Here, $\sigma(\cdot)$ and $f(\cdot)$ indicate the activation values of σ and f. In this article, we abstract a general and simple subtraction form $f - \sigma$, where σ is the same type of nonlinear function as f. $(0, f)$ is a subset of the value range of σ; so this coupled design would be a sufficient condition to achieve the subtraction gate.

Compared with the multiplication gates used in LSTM, the subtraction gate is a more direct way to control the transfer of information, because the information is directly subtracted from the current neurons. The activation value of σ in the subtraction gates is not limited to $(0, 1)$, but σ corresponds to f. Thus, the options for f and σ include logistic sigmoid, hyperbolic tangent, or identity functions. Recurrent networks with the proposed subtraction gates are called SRNNs. As σ is not fixed, the activation function settings of SRNNs are more flexible than those of LSTM.

We directly replace the multiplication terms with subtraction terms to control the input and output. However, when applying the proposed subtraction gates within an SRNN, it is important to consider the exact form of σ and f. The general form of the memory cell with subtraction gate is

$$s_i(t+1) = s_i(t) - \sigma_F(F_i(t+1)) + g(Z_i(t+1)) - \sigma_I(I_i(t+1)). \quad (13)$$

As the memory cells of LSTM are linear self-connections, we design the subtraction forget gate by considering the expansion formula of $s_i(t)$ as follows:

$$s_i(t) = s_i(t-1) + \underline{g(Z_i(t)) - \sigma_I(I_i(t))}. \quad (14)$$

The memory cells accumulate network activations with input gates. The term with underline has the same form with output $y_i(t)$ as follows:

$$y_i(t) = \underline{h(s_i(t)) - \sigma_O(O_i(t))}. \quad (15)$$

Therefore, we replace the multiplication forget gates by directly subtracting the last output. This design satisfies the design philosophy of subtraction gates because the subtraction gate has the same form as the linear self-connections of memory cells. In this design, we do not involve new parameters in the subtraction term $s_i(t) - y_i(t)$. The total forward pass of SRNN is formulated as follows:

$$\begin{cases} Z_i(t+1) = \sum_j w_{ij} y_j(t) + \sum_k w_{ik} x_k(t) \\ I_i(t+1) = \sum_j w^I_{ij} y_j(t) + \sum_k w^I_{ik} x_k(t) \\ O_i(t+1) = \sum_j w^O_{ij} y_j(t) + \sum_k w^O_{ik} x_k(t) \\ s_i(t+1) = s_i(t) - y_i(t) \\ \qquad + g(Z_i(t+1)) - \sigma_I(I_i(t+1)) \\ y_i(t+1) = h(s_i(t+1)) - \sigma_O(O_i(t+1)). \end{cases} \quad (16)$$

Stable Internal State Without Drift: Under the design philosophy of subtraction gates, the term $f - \sigma$ has a larger value range than $f \odot \sigma$. The subtraction gates may produce a negative value: $f - \sigma < 0$. The larger value range enhances the power of subtraction gates to keep stable internal states. As we know, LSTM has the internal state drift problem because of the limited ability that input gates cannot prevent network inputs from being mostly positive or mostly negative. The internal state drift will induce gradient vanishing because the term $h'(s(t))$ will have an infinitely small value.

In LSTM, the value shift of $g \odot \sigma_I$ is highly relative with g, while σ_I controls the number of information flow. At the beginning of the training stage, g is highly relative to the distribution of training data with randomly initialized parameters. For example, if the mean of training data is greater than 0, the internal state $s(t)$ tends to be a large positive value (refer to Section V-A). Therefore, the internal state $s(t)$ is easy to be mostly positive or negative in LSTM. To solve this problem, Hochreiter and Schmidhuber [10] proposed to initially bias the input gate toward zero by carefully designing the bias weights, and this is very important when the time lag is long. SRNN does not have this problem at all. The value shift of $g - \sigma_I$ is not relative to the training data because g and σ_I are coupled. At the beginning of the training stage, the internal stage can keep stable with randomly initialized parameters. A typical experiment is the adding problem. We perform the adding experiment in Section V, and we observe that SRNN achieves better performance than LSTM without carefully initializing the input gate's bias weights.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

B. Gradient Flow of Subtraction Recurrent Neural Networks

In Section IV-A, we gave a plausible explanation of how the subtraction gates work in terms of the forward pass of SRNNs. In this section, we consider the gradient flow of SRNNs during training. The partial derivative of $\mathcal{J}_{\text{total}}$ with respect to the recurrent weight matrixes $\mathbf{W}_{\text{rec}}$ is

$$\frac{\partial \mathcal{J}_{\text{total}}}{\partial W_{\text{rec}}} = \sum_{t \geqslant 1} \sum_{1 \leqslant \tau \leqslant t} \frac{\partial \mathcal{J}(t)}{\partial \mathbf{y}(t)} \cdot \frac{\partial \mathbf{y}(t)}{\partial \mathbf{y}(\tau)} \cdot \frac{\partial \mathbf{y}^{+}(\tau)}{\partial W_{\text{rec}}} \tag{17}$$

where $\mathbf{W}_{\text{rec}} \in \{\mathbf{W}_Z, \mathbf{W}_I, \mathbf{W}_O\}$. Z, I, and O indicate the pre-activations of recurrent neurons, input neurons, and output neurons, respectively.

Unlike standard RNNs, the gradient computation of SRNNs cannot use the chain rule of (6), because the network still contains memory cells. In [10], the truncation terms

$$\begin{cases} \dfrac{\partial Z(t)}{\partial \mathbf{y}(t-1)} \approx_{\text{tr}} 0 \\ \dfrac{\partial I(t)}{\partial \mathbf{y}(t-1)} \approx_{\text{tr}} 0 \\ \dfrac{\partial O(t)}{\partial \mathbf{y}(t-1)} \approx_{\text{tr}} 0 \end{cases} \tag{18}$$

are utilized to analyze the gradient flow of LSTM. We expand the partial derivative of $((\partial \mathbf{y}(t))/(\partial \mathbf{y}(\tau)))$ at step $t-1$ as

$$\frac{\partial \mathbf{y}(t)}{\partial \mathbf{y}(\tau)} = h'(\mathbf{s}(t)) \cdot \frac{\partial \mathbf{s}(t)}{\partial \mathbf{y}(\tau)} + \sigma'_O(O(t)) \cdot \frac{\partial O(t)}{\partial \mathbf{y}(t-1)} \cdot \frac{\partial \mathbf{y}(t-1)}{\partial \mathbf{y}(\tau)}. \tag{19}$$

Motivated by the analysis of the gradient flow in LSTM, we truncate the error flowing into the output by $((\partial O(t))/(\mathbf{y}(t-1))) \approx_{\text{tr}} 0$. Hence, the approximation of $((\partial \mathbf{y}(t))/(\partial \mathbf{y}(\tau)))$ is

$$\begin{aligned} \frac{\partial \mathbf{y}(t)}{\partial \mathbf{y}(\tau)} &\approx_{\text{tr}} h'(\mathbf{s}(t)) \cdot \frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(\tau+1)} \cdot \frac{\partial \mathbf{s}(\tau+1)}{\partial \mathbf{y}(\tau)} \\ &\approx h'(\mathbf{s}(t)) \prod_{\tau+1 < \epsilon \leqslant t} \frac{\partial \mathbf{s}(\epsilon)}{\partial \mathbf{s}(\epsilon-1)} \cdot \frac{\partial \mathbf{s}(\tau+1)}{\partial \mathbf{y}(\tau)}. \end{aligned} \tag{20}$$

Redefine $\mathbf{A} := ((\partial \mathbf{s}(\epsilon))/(\partial \mathbf{s}(\epsilon-1)))$, the Jacobian is

$$\begin{aligned} \mathbf{A} = \mathbf{I} &- \text{diag}[h'(\mathbf{s}(\epsilon-1))] \\ &+ \text{diag}[g'(Z(\epsilon))] \cdot \frac{\partial Z(\epsilon)}{\partial \mathbf{y}(\epsilon-1)} \cdot \frac{\partial \mathbf{y}(\epsilon-1)}{\partial \mathbf{s}(\epsilon-1)} \\ &- \text{diag}\big[\sigma'_I(I(\epsilon))\big] \cdot \frac{\partial I(\epsilon)}{\partial \mathbf{y}(\epsilon-1)} \cdot \frac{\partial \mathbf{y}(\epsilon-1)}{\partial \mathbf{s}(\epsilon-1)}. \end{aligned} \tag{21}$$

Applying the truncation $((\partial Z(\epsilon))/(\partial \mathbf{y}(\epsilon-1))) \approx_{\text{tr}} 0$ and $((\partial I(\epsilon))/(\partial \mathbf{y}(\epsilon-1))) \approx_{\text{tr}} 0$, we could obtain the gradient

$$\mathbf{A} = \mathbf{I} - \text{diag}[h'(\mathbf{s}(\epsilon-1))]. \tag{22}$$

Similar to the forget multiplication gates of LSTM, the subtraction gates applied in memory cells can destroy the constant error flow during training. However, there is a balance between resetting the memory cells and keeping a constant error flow. Regardless, SRNNs provide a way to adjust the norm of the Jacobian matrix $\mathbf{A}$. When $h'(s_i(\epsilon-1)) \to 0$, the eigenvalue λ_i of $\mathbf{A}$, $i \in \{1, 2, \ldots, n\}$ is close to 1, which is the optimal situation for learning long-term dependencies.

Therefore, we initialized $\text{diag}[h'(\mathbf{s}(\epsilon-1))]$ to be close to the $\mathbf{0}$ matrix before training SRNNs. In LSTM, a bias is usually added to the multiplication forget gates when initializing the parameters. From our experiments, we observe that adding or subtracting a bias in the pre-activation Z_i of the SRNN would speed up the training. Moreover, we note that the Relu function would strongly destroy the constant error flow of memory cells because the derivative of Relu function is equal to 1 when the pre-activation satisfies that $s(t) > 0$. We will evaluate if the Relu function works in the ERG experiments.

C. Learning Algorithm of Subtraction Recurrent Neural Networks

The major learning algorithms of RNNs are RTRL and BPTT [12]. BPTT views the RNNs as unrolled FNNs and accumulates gradients through times to update the RNN parameters. BPTT is effective and easily implemented, but struggles with continual sequence learning problems. RTRL has the potential to solve this problem by updating the parameters after every time step.

To reduce the computational complexity and improve the performance analysis, a truncated version of RTRL was employed in [10]. Alternatively, a BPTT-based algorithm with full gradients can be used to train the LSTM without harming its performance [29]. In recent years, computation frameworks, such as TensorFlow[1], have made full gradient computing based on BPTT very easy to implement.

In this section, we compute the full gradients and truncated version to train our SRNNs based on RTRL. To decrease the number of parameters in LSTM, several memory cells can be set to consist of only one block [10]. In the proposed method, we do not use blocks because they are not the major factor in simplifying the network structure. The forward computing of SRNNs is shown in Fig. 2.

Given the loss $\mathcal{J}(t)$ at time t, RTRL attempts to update w_{ij}^Z, w_{ij}^I, and w_{ij}^O by computing $((\partial \mathcal{J}(t))/(\partial w_{ij}^Z))$, $((\partial \mathcal{J}(t))/(\partial w_{ij}^I))$, and $((\partial \mathcal{J}(t))/(\partial w_{ij}^O))$, respectively. In this section, we only compute the gradients of recurrent weights, because the gradient computation of input weights is similar with recurrent weights. Let the partial derivative of $\mathcal{J}(t)$ with respect to output $y_i(t)$ be $\delta_i(t)$. We define

$$\vartheta_{kij}^{\alpha}(t) = \frac{\partial y_k(t)}{\partial w_{ij}^{\alpha}} \tag{23}$$

where $\vartheta_{kij}^{\alpha}(t)$ indicates the partial derivative of $y_k(t)$ with respect to w^{α}, α denotes the state of input, output, or memory cells and $\alpha \in \{Z, I, O\}$. The corresponding gradients are calculated as

$$\begin{aligned} \frac{\partial \mathcal{J}(t)}{\partial w_{ij}^{\alpha}} &= \sum_k \frac{\partial \mathcal{J}(t)}{\partial y_k(t)} \cdot \frac{\partial y_k(t)}{\partial w_{ij}^{\alpha}} \\ &= \sum_k \frac{\partial \mathcal{J}(t)}{\partial y_k(t)} \cdot \vartheta_{kij}^{\alpha}(t). \end{aligned} \tag{24}$$

[1] https://www.tensorflow.org

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

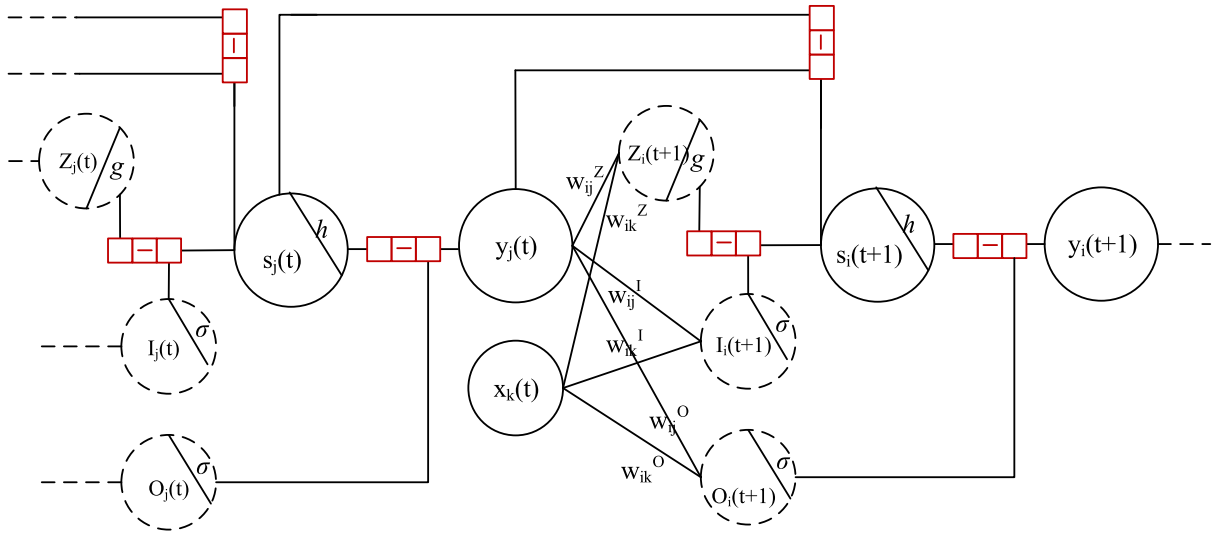


Fig. 2. Illustration of SRNNs. Red circles indicates the elementwise subtraction operations. As illustrated, the subtraction can be applied in the input, output, and memory cells.

Using a similar chain rule as for RTRL, we can calculate the full gradients as

$$\vartheta^Z_{kij}(t) = h'(s_k(t)) \cdot \frac{\partial s_k(t)}{\partial w^Z_{ij}} - \sigma'_O(O_k(t)) \cdot \sum_m w^O_{km} \vartheta^Z_{mij}(t-1) \tag{25}$$

$$\vartheta^I_{kij}(t) = h'(s_k(t)) \cdot \frac{\partial s_k(t)}{\partial w^I_{ij}} - \sigma'_O(O_k(t)) \cdot \sum_m w^O_{km} \vartheta^I_{mij}(t-1) \tag{26}$$

and

$$\vartheta^O_{kij}(t) = h'(s_k(t)) \cdot \frac{\partial s_k(t)}{\partial w^O_{ij}} - \sigma'_O(O_k(t)) \times \left(\sum_m w^O_{km} \vartheta^O_{mij}(t-1) + \mu_{ki} y_j(t-1) \right). \tag{27}$$

The corresponding truncated equations are

$$\begin{cases} \vartheta^Z_{kij}(t) \approx_{\text{tr}} h'(s_k(t)) \cdot \dfrac{\partial s_k(t)}{\partial w^Z_{ij}} \\ \vartheta^I_{kij}(t) \approx_{\text{tr}} h'(s_k(t)) \cdot \dfrac{\partial s_k(t)}{\partial w^I_{ij}} \\ \vartheta^O_{kij}(t) \approx_{\text{tr}} h'(s_k(t)) \cdot \dfrac{\partial s_k(t)}{\partial w^O_{ij}} \\ \qquad - \sigma'_O(O_k(t)) \cdot \mu_{ki} y_j(t-1). \end{cases} \tag{28}$$

We define

$$p^\alpha_{kij}(t) = \frac{\partial s_k(t)}{\partial w^\alpha_{ij}} \tag{29}$$

to indicate the partial derivative of the memory cells $s_k(t)$ with respect to w^α_{ij}. The partial derivative $p^\alpha_{kij}(t)$ can be calculated as

$$p^Z_{kij}(t) = p^Z_{kij}(t-1) - \vartheta^Z_{kij}(t-1) + g'(Z_k(t)) \left(\sum_m w^Z_{km} \vartheta^Z_{mij}(t-1) + \mu_{ki} y_j(t-1) \right) - \sigma'_I(I_k(t)) \cdot \sum_m w^I_{km} \vartheta^Z_{mij}(t-1) \tag{30}$$

$$p^I_{kij}(t) = p^I_{kij}(t-1) - \vartheta^I_{kij}(t-1) + g'(Z_k(t)) \cdot \sum_m w^Z_{km} \vartheta^I_{mij}(t-1) - \sigma'_I(I_k(t)) \left(\sum_m w^I_{km} \vartheta^I_{mij}(t-1) + \mu_{ki} y_j(t-1) \right) \tag{31}$$

and

$$p^O_{kij}(t) = p^O_{kij}(t-1) - \vartheta^O_{kij}(t-1) + g'(Z_k(t)) \cdot \sum_m w^Z_{km} \vartheta^O_{mij}(t-1) - \sigma'_I(I_k(t)) \cdot \sum_m w^I_{km} \vartheta^O_{mij}(t-1). \tag{32}$$

The corresponding truncated equations are

$$p^Z_{kij}(t) \approx_{\text{tr}} p^Z_{kij}(t-1) - \vartheta^Z_{kij}(t-1) + g'(z_k(t)) \cdot \mu_{ki} y_j(t-1) \tag{33}$$

$$p^I_{kij}(t) \approx_{\text{tr}} p^I_{kij}(t-1) - \vartheta^I_{kij}(t-1) - \sigma'_I(I_k(t)) \cdot \mu_{ki} y_j(t-1) \tag{34}$$

and

$$p^O_{kij}(t) \approx_{\text{tr}} p^O_{kij}(t-1) - \vartheta^O_{kij}(t-1). \tag{35}$$

We start training the SRNN by initializing $y(0) = 0$, $s(0) = 0$, $\vartheta^\alpha(0) = 0$, and $p^\alpha(0) = 0$ for all $\alpha \in \{Z, I, O\}$. All weight matrices are initialized randomly, and a bias of 1

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Learning Algorithm Flowchart of SRNN

Input : Max iteration T, No. of recurrent neurons n, learning rate λ, initial weights, inputs, and states w_{ij}^{α}, $y_i(0)$, $s_i(0)$, $x_i(0)$, $p_{kij}^{\alpha}(0)$, $\vartheta_{kij}^{\alpha}(0)$, where $i, j, k \leq n$, $\alpha \in \{Z,\ I,\ O\}$.

Output: Updated weights w_{ij}^{α}.

for $t = 1, 2, \ldots, T$ *Compute loss* $\mathcal{J}(t)$;
  for $k = 1, 2, \ldots, n$ **do**
    Compute $I_k(t)$, $Z_k(t)$, $O_k(t)$, $s_k(t)$ *according to Equation (16);*
    for $i = 1, 2, \ldots, n$ **do**
      for $j = 1, 2, \ldots, n$ **do**
        Compute $p_{kij}^{\alpha}(t)$ *according to Equation (30-32) or Equation (33-35);*
        Compute $\vartheta_{kij}^{\alpha}(t)$ *according to Equation (25-27) or Equation (28);*
      end
    end
  end
end
for $i = 1, 2, \ldots, n$ **do**
  for $j = 1, 2, \ldots, n$ **do**
    Compute gradients $\frac{\partial \mathcal{J}(t)}{\partial w_{ij}^{\alpha}}$ *according to Equation (24);*
    Update parameters as follows: $w_{ij}^{\alpha} = w_{ij}^{\alpha} - \lambda \cdot \frac{\partial \mathcal{J}(t)}{\partial w_{ij}^{\alpha}}$;
  end
end
do
end
return w_{ij}^{α}.

is added to or subtracted from $Z_i(0)$. To sum up, we conclude the learning algorithm flowchart in **Algorithm 1**.

V. Experiments

In this article, we perform five experiments. The first one is the adding problem. We attempt to prove that SRNN can dynamically tune the internal state without carefully initializing the input gate. The second one is the ERG. This experiment is a profound typical simulation introduced in the original LSTM article [10], and it has become a basic experiment in some literature, for instance, in [16] and [20]. The third experiment is the PTB. The PTB is a word-level prediction language modeling experiment. We adopt the standard data set following [30]. The fourth experiment is Pixel-by-Pixel MNIST. Classifying the MNIST digits using RNNs is a challenging sequence learning problem. The final experiment is the Yelp2013, a document classification experiment. We perform ablation studies in ERG and PTB and compare SRNNs with baseline methods in Pixel-by-Pixel MNIST and Yelp2013. Note that training an SRNN is faster than training an LSTM because the subtraction gate applied in the memory cells involves no parameters, but the LSTM involves the same number of parameters with memory cells. We have verified this merit of SRNNs' time complexity on PTB and Pixel-by-Pixel experiments.

A. Adding Problem

The adding problem is a challenging sequence regression task, and this task cannot be solved by random search, and other recurrent learning algorithms except LSTM [10]. LSTM solved this problem after training at least 74 000 epochs by carefully setting the input gate's bias weights.

The task can be explained as follows: each input sequence has an element consisting of two components. The first component is a real value by chosen from the interval $[-1, 1]$ randomly, and the second component is an element chosen from the set of $\{-1.0, 0.0, 1.0\}$, which acts as the marker. When the second components are equal to 1.0, the sum of the first components in corresponding pairs is output at the end of each sequence. The sequence length is a randomly chosen integer between T and $T + (T/10)$. Each sequence is marked with the following rules: one of the first ten pairs is randomly selected and marked (the second component is equal to 1.0), whose first component is called X_1. Then, one of the first $(T/2) - 1$, which is unmarked in the first step, is selected and marked, and its first component is called X_2. The second components of the remaining pairs are zero except for the first and final pair, whose second components are -1.0. The target is $0.5 + ((X_1 + X_2)/4.0)$. A sequence is predicted correctly if the error is below 0.04.

Training Details: We compare the performance of LSTM and SRNN with two input neurons, one output neuron, and five recurrent neurons. In LSTM, h is a sigmoid in $[-1, 1]$ and g is a sigmoid in $[-2, 2]$, while in SRNN, all are sigmoid functions. The learning rate is 0.5. The training is stopped if the average training error is below 0.01, and the correct predicted sequence number is larger than 2000. The test set consists of 2560 randomly chosen sequences. All network weights are randomly initialized in the range $[-0.1, 0.1]$ except that LSTM's input gate bias is initialized with negative values (the first three bias weights are -3, and the left two bias weights are -6).

The experiments are implemented with MATLAB R2016a on an i7 3.4-GHz dual-core PC with 16-GB CPU. The experimental results are present in Table I, where each result is a mean of 10 trials. It can be concluded that the SRNN has a better performance than LSTM from the experimental results. We also plot the training errors of SRNN and LSTMs with different input gate biases in Fig. 3, where $T = 100$. We observe that SRNN converges faster than LSTM because LSTM has more stable internal states than LSTM.

B. Embedded Reber Grammar

ERG is a popular sequence learning experiment [10], [16]. Following certain grammar rules, ERG generates sequences of different lengths from the characters {"B," "P," "T," "S," "X," "V," "E"}. Sequences are generated with some internal state transition probability. In this article, we set the transition probability of all branching paths to 0.5. We train the networks to read one symbol of these string sequences and predict

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

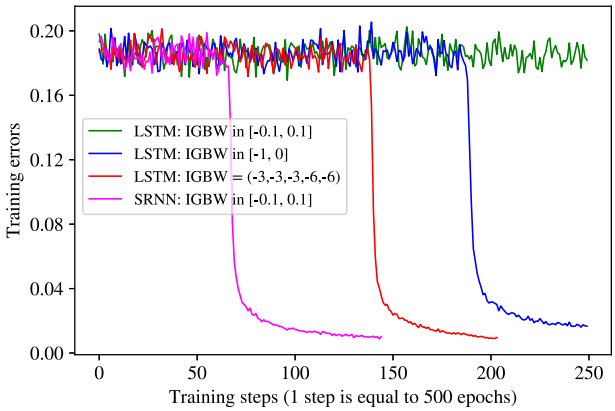


Fig. 3. Training errors with different Input Gate Biase Weights, where $T = 100$.

TABLE I
EXPERIMENTAL RESULTS OF ADDING PROBLEM

Model	T	Wrong predictions	Success after
LSTM	100	2.3 of 2560	84335.6
	500	2 of 2560	877259.0
	1000	1.4 of 2560	3601574.2
SRNN	100	2.8 of 2560	79563.2
	500	0.8 of 2560	604257.9
	1000	2.1 of 2560	2967134.5

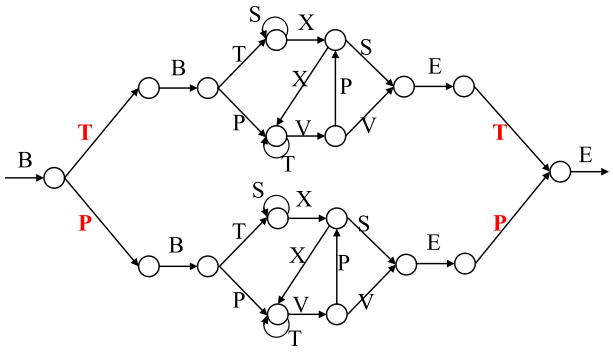


Fig. 4. Illustration of ERG. The learning task requires the model to have long-time memory, as the final red bold "T" or "P" state are determined by the first red bold "T" or "P" state.

the next symbol. Details about the grammar rules are shown in Fig. 4.

1) Data Set: We use the one-hot representation of seven neurons to represent a symbol. The training set is generated randomly and includes 8000 sequences. We generate the test set at random, but if a sequence has already been used for training, we regenerate the sequence. A total of 256 test strings are generated with an average length of 35 characters, which is longer than that of the training set. The generation process of the test set guarantees long-term dependencies.

2) Implementation: The experiments were implemented in MATLAB on an i7 3.4-GHz dual-core PC. The SRNN learning algorithm was described in Section IV-C and the LSTM learning algorithm is the truncated algorithm of [10]. Fifteen recurrent neurons were used in all trials. An output layer with seven neurons was stacked on each RNN, and the mean-square error cost function was used for training. The parameters of all networks were initialized to values in $[-0.1, 0.1]$.

To validate the impact of nonlinear functions, such as g, h, and σ, we first conducted ten trials to test the *hyperbolic tangent*, *logistic sigmoid*, or *Relu* functions. As we explained in Section IV-A, the subtraction gate performs directly as a coupled function, which means g has the same function settings as σ_I and h has the same function settings as σ_O. The test consisted of six trials with coupled functions and four trials without. The learning rate of all trials was set to 0.5.

From the test accuracy curves of Fig. 5, we observe that five trials with coupled functions achieve test accuracies of 100%, and the models convergent faster than those without coupled functions. These trials validate our explanation of the subtraction gate mechanism. We also find that the SRNNs setting h and σ_O to the *Relu* function failed (green curve in Fig. 5). This observation verifies our analyze of the *Relu* function in Section IV-B. The couple *hyperbolic tangent* or *logistic sigmoid* functions work well in the curves.

Moreover, we tested a total of 6 RNNs: 1) the normal LSTM with forget gates; 2) LSTM without forget gates (IO-LSTM); 3) SRNN with subtraction applied only to the input (I-SRNN); 4) SRNN with subtraction applied only to the output (O-SRNN); 5) SRNN with subtraction applied to both input and output (IO-SRNN); and 6) normal SRNN with subtraction applied to the input, output, and memory cells. All subtraction-based models used coupled *hyperbolic tangent* functions and were tested a total of 100×; the number of successful trials was counted and recorded. We calculated the test accuracy after every 100 training sequences and terminated the training procedure when the test accuracy reached 100%. When the network could not achieve 100% accuracy after 8000 train sequences, we terminated the training procedure and recorded the trial as unsuccessful in calculating the average number of successful training epochs. The learning rate was initialized to 0.5 and decreased by multiplying a factor of 0.98 after every 100 training epochs.

The results in Table II indicate that SRNNs offer better performances than LSTMs. Among them, results of LSTM, IO-SRNNs, and SRNNs achieve full success in 100 trials, which indicates that both multiplication and subtraction gates boost the performance. From accounting of the average number of successful training epochs, we find that subtraction gates converge faster than multiplication gates. Comparing the results of IO-LSTM, I-SRNNs, O-SRNNs, and IO-SRNNs, we find that subtraction gates perform better than multiplication gates when gates are applied in controlling how much information are input and output from memory cells.

C. Penn Tree Bank

The PTB is a large corpus containing 929 K training words, 73 K validation words, and 82 K test words. We repeated the word-level prediction language modeling experiments of [30]. The samples were split every 20 words. In this experiment, we implemented the SRNNs using the TensorFlow framework and adopted the BPTT algorithm for training. We do ablation

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

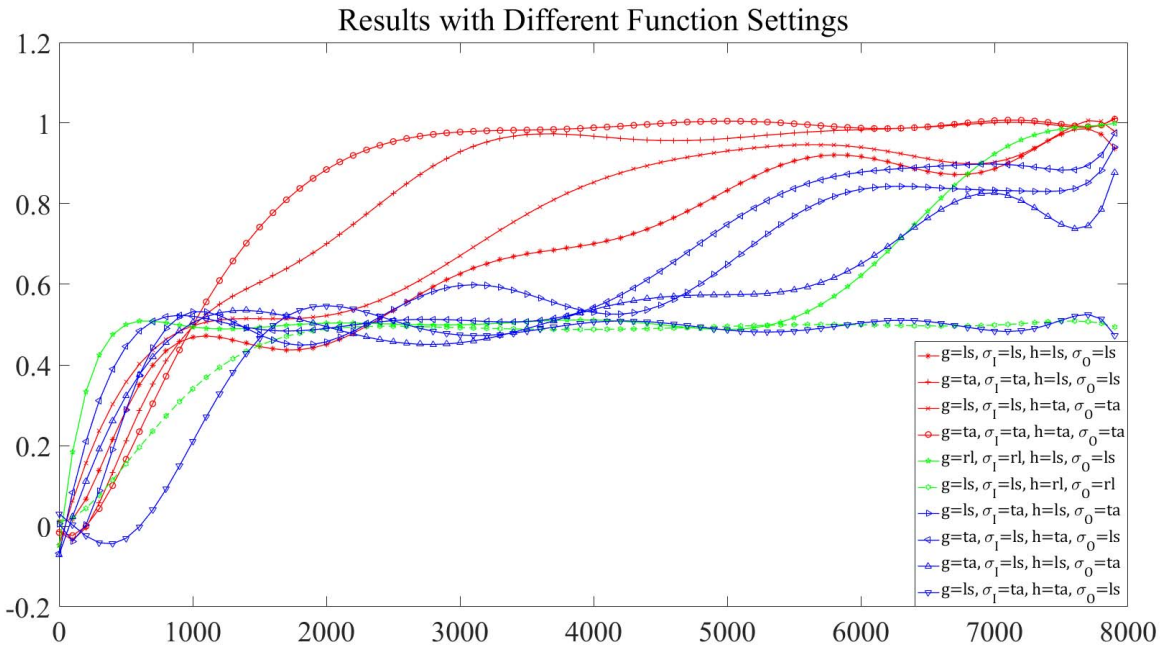


Fig. 5. Results for SRNNs with different function settings. The x-axis indicates the number of training sequences and the y-axis indicates the accuracy of the test set. The red lines indicate trials in which the activation and subtraction functions were coupled, the green lines indicate activation and subtraction functions with coupled *Relu*, and the blue lines indicate the inverse. "ta" denotes the hyperbolic *tangent* function, "ls" denotes the *logistic sigmoid* function, and "rl" denotes the *Relu* function.

TABLE II

RESULTS FOR LSTMS AND SRNNS WITH DIFFERENT STRUCTURES. THE RESULTS OF 100 TRIALS WERE COUNTED AND THE BEST RESULTS ARE SHOWN IN BOLD

Methods	Models	Success trials	Success After
Gate	LSTM	**100**	3928
	IO-LSTM	78	5485
Subtraction	I-SRNNs	99	3685
	O-SRNNs	85	4965
	IO-SRNNs	**100**	3439
	SRNNs	**100**	**3336**

TABLE III

EXPERIMENTAL RESULTS USING THE NINE NETWORKS. BETTER RESULTS ARE SHOWN IN BOLD. "RN" INDICATES RECURRENT NEURONS

Index	Models	≈ 261.7K parameters		200 RNs	
		No. of RNs	Perplexity	No. of parameters	Perplexity
1	RNN	420	129.01	80.8K	137.13
2	LSTM	200	114.32	261.7K	114.32
3	GRU	235	167.09	201.6K	190.26
4	F-I-SRNN	294	157.18	121.0K	155.61
5	F-O-SRNN	294	**105.93**	121.0K	**107.04**
6	F-SRNN	235	111.24	201.6K	111.24
7	I-SRNN	294	124.48	121.0K	122.00
8	O-SRNN	294	119.72	121.0K	124.19
9	SRNN	235	113.41	201.6K	115.20

studies of different RNN models but not attempt to achieve state-of-the-art. Therefore, following [30], all experiments used mini-batches of 20. The learning rate was initialized to 1 and decreased by a factor of 2 after each of 13 training epochs.

To validate the performance of our methods, we tested SRNNs with different subtraction settings. We first compared nine variants with 200 recurrent neurons in each layer, and then compare them with about 261.7 K parameters by varying the number of neurons. These networks required an average of 2–3-h training time using an NVIDIA K40 GPU. All experimental results are averages over five trials. For the gate terms, σ was a logistic sigmoid function, whereas, for the subtrahend terms, σ was a hyperbolic tangent function, the same as f. The nine variants were as follows.

1) *RNN:* Normal RNN without gates and subtractions.
2) *LSTM:* LSTM structure with input, output, and forget gates.
3) *GRU:* Normal GRU.
4) *F-I-SRNN:* A hybrid model with multiplication forget gates and subtraction applied only to the input.
5) *F-O-SRNN:* A hybrid model with multiplication forget gates and subtraction applied only to the output.
6) *F-SRNN:* A hybrid model with multiplication forget gates and subtraction applied to both input and output.
7) *I-SRNN:* SRNN with subtraction applied to the input and memory cells.
8) *O-SRNN:* SRNNs with subtraction applied to the output and memory cells.
9) *SRNN:* Normal SRNNs with full subtraction deployment.

The experimental results are presented in Table III, where the numbers of recurrent neurons and parameters are listed. Comparing the results of models 4, 5, and 6, we find that the input subtraction gate is incompatible with multiplication forget gates, but models combining the output subtraction

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

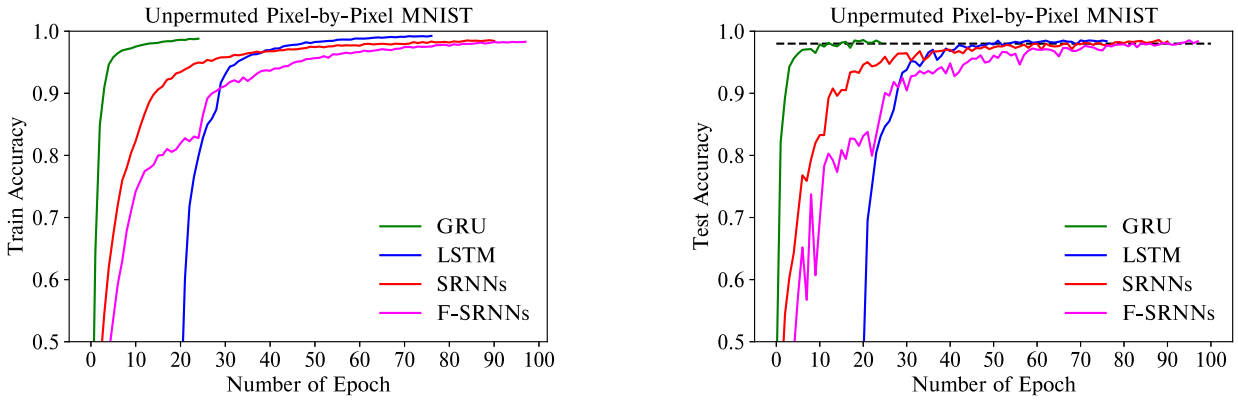


Fig. 6. Training and test accuracies of unpermuted Pixel-by-Pixel MNIST.

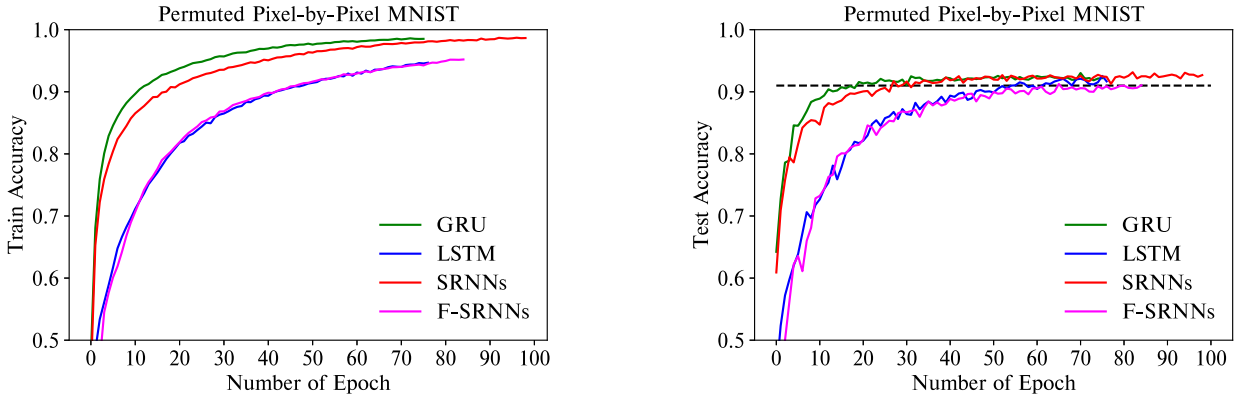


Fig. 7. Training and test accuracies of permuted Pixel-by-Pixel MNIST.

gate and forget multiplication gate have the best performance. The results of models 2 and 9 suggest that the proposed SRNNs achieve comparable performance to LSTM. However, the results of models 5 and 6 indicate that F-SRNNs outperform LSTM in the PTB task. As found in [18], GRU does not work well in language modeling tasks.

D. Pixel-by-Pixel MNIST

Classifying the MNIST digits using RNNs is a challenging problem because the image pixels are input sequentially. The MNIST images contain 784 pixels, which makes the learning a long-term memory problem. The learning task is to input a fixed-length sequence and output the category as one of ten classes, which is called Pixel-by-Pixel MNIST [27], [31]. In the permuted version, the pixels are randomly shuffled, so the RNNs have to learn nonlocal dependencies.

In this article, we use standard MNIST data: 60 000 training examples and 10 000 testing examples. The cross-entropy loss function and early stopping were applied. We tested four RNN models, including the widely used baseline models of GRU and LSTM, and our proposed SRNNs and F-SRNNs. All models used the Adam optimizer for training. With 128 recurrent neurons, the learning rate of the SRNNs and F-SRNNs was set to 0.0005 and that of GRU and LSTM was 0.001; with 512 recurrent neurons, the learning rate of all models was set to 0.0001. The batch size and number of recurrent neurons were fixed to 32 and 128, respectively. The gradient clipping parameter was set to 1 for all cases. The code was implemented with TensorFlow.

The experimental results are reported in Table IV and the accuracy curves are plotted in Figs. 6 and 7. The results plotted in Figs. 6 and 7 are those with 128 recurrent neurons. From the curves, we find that GRU converges faster than the other models. In the unpermuted MNIST experiments, LSTM requires approximately 20th training epochs, whereas the SRNNs and F-SRNNs display smooth performance curves. In the permuted MNIST experiments, GRU converges faster than other methods in the training stage. However, with the early stop strategy, the SRNNs and F-SRNNs achieve comparable test accuracies to those of LSTM and GRU. Overall, GRU is the best choice for unpermuted Pixel-by-Pixel MNIST because it achieves an accuracy of 98.95%.

We also tested our model with 512 recurrent neurons for comparison with unitary RNNs (uRNNs) and full-capacity uRNNs. Our proposed SRNNs achieve up to 98.92% test accuracy. In the permuted MNIST experiments, the SRNNs attain an accuracy of 94.29%, the best test accuracy. Full-capacity uRNN performs well but is not suitable for unpermuted MNIST experiments. To sum up, the proposed subtraction-based models offer comparable performance with GRU and outperform LSTM in the Pixel-by-Pixel MNIST experiments.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

EXPERIMENTAL RESULTS FOR PIXEL-BY-PIXEL MNIST. TEST ACCURACIES ARE COMPUTED AFTER THE TRAINING STOPPED. *lr* DENOTES LEARNING RATE AND *N* DENOTES THE NUMBER OF RECURRENT NEURONS. THE BEST RESULTS ARE SHOWN IN BOLD AND THE SECOND ARE WITH UNDERLINE

	Models	N	lr	Test Acc
Unpermuted	uRNN [27]	512	-	95.10%
	Full-capacity uRNN [31]	116	-	92.80%
	Full-capacity uRNN [31]	512	-	96.90%
	LSTM	128	0.001	98.52%
	GRU	128	0.001	**98.95%**
	SRNN	128	0.0005	98.22%
	F-SRNN	128	0.0005	98.55%
	SRNN	512	0.0001	<u>98.92%</u>
	F-SRNN	512	0.0001	98.52%
Permuted	uRNN [27]	512	-	91.40%
	Full-capacity uRNN [31]	116	-	92.10%
	Full-capacity uRNN [31]	512	-	<u>94.10%</u>
	LSTM	128	0.001	92.34%
	GRU	128	0.001	93.03%
	SRNN	128	0.0005	92.71%
	F-SRNN	128	0.0005	91.35%
	SRNN	512	0.0001	**94.29%**
	F-SRNN	512	0.0001	92.71%

TABLE V

EXPERIMENTAL RESULTS FOR YELP2013

Models		Accuracy
Baseline	CNNs[32]	59.70%
	Convolutional gated RNN[32]	63.70%
	LSTM gated RNN[32]	65.10%
	75D Hierarchical attention networks[34]	68.20%
	100D Simple Attention (LSTM)[33]	67.70%
	100D Structured Attention (LSTM)[33]	68.60%
Our reimplementation	100D Structured Attention (LSTM)	68.56%
	100D Structured Attention (GRU)	68.42%
	100D Structured Attention (SRNN)	68.77%
	100D Structured Attention (F-SRNN)	**68.82%**

E. Yelp2013 Document Classification

Yelp2013 data set was obtained from the 2013 Yelp Dataset Challenge. It contains 335 018 documents with human ratings from 1 to 5 (five classes) scores, which are used as the gold labels. We follow the setting of [32] and [33] to split data set into partitions (80/10/10) for training, development, and testing. We implement the LSTM, GRU, SRNN, and F-SRNN under the architecture of the structured attention method [33]. The word embeddings with dimensions of 200 are initialized by training word2vec on the training and development set. The hidden size of all networks is 100. The weight decay, learning rate, and dropout rate are $1e^{-4}$, 0.05, and 0.3, respectively.

We give the results in Table V. The reimplemented structured attention method with LSTM achieves -0.04% decrease with respect to accuracy with [33]. The structured attention methods with SRNN and F-SRNN achieve $+0.17\%$ and $+0.22\%$ increase with respect to accuracy. The performance improvements indicate that our proposed SRNN and F-SRNN models are suitable backbone networks for the structure attention method.

VI. CONCLUSION

In this article, we have proposed a novel subtraction-based gate mechanism for learning long-term dependencies. We described the forward computation and learning algorithm of the proposed SRNNs and analyzed their ability to deal with the gradient vanishing/explosion problem. The SRNNs were validated through a series of ERG, PTB, and Pixel-by-Pixel MNIST experiments and achieved comparable performance to conventional LSTM and GRU models. F-SRNNs (a hybrid of forget multiplication gates and SRNNs) display the potential to be great to learn long-term memory.

Different from multiplication gate RNN models, SRNNs have more flexible activation function settings, but the widely used Relu function cannot be used because of its constant derivative. In [35], a simple strategy was proposed for the use of *Relu* function in RNNs, whereby the recurrent weight matrix is initialized as the identity matrix. In our future work, we will attempt to improve the performance of SRNNs and F-SRNNs through this identity matrix initializing strategy.

It can be seen from our expanded experiments that it is difficult to train a network of stacked SRNNs. We have done some experiments both on the networks of stacked two or three SRNNs and LSTMs, respectively; it seems that the stacked LSTM is better than the SRNN in this particular situation. In [36], multilayer LSTM achieved the state-of-the-art with the help of dropout. It is not clear so far why stacked SRNNs perform not well. Stacking neural networks maybe not a good mechanism to form a deep neural network for SRNNs. However, we will further study this problem in future work.

REFERENCES

[1] A. Graves, A.-R. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2013, pp. 6645–6649.

[2] J. Chung, K. Cho, and Y. Bengio, "A character-level decoder without explicit segmentation for neural machine translation," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, vol. 1, 2016, pp. 1693–1703.

[3] A. Graves and J. Schmidhuber, "Offline handwriting recognition with multidimensional recurrent neural networks," in *Proc. 22nd Annu. Conf. Neural Inf. Process. Syst. (NIPS)*, 2008, pp. 545–552.

[4] Z. Yi, "Foundations of implementing the competitive layer model by Lotka–Volterra recurrent neural networks," *IEEE Trans. Neural Netw.*, vol. 21, no. 3, pp. 494–507, Mar. 2010.

[5] Y. Chen, L. Zhang, and Z. Yi, "Nearest convex hull classification by using Lotka–Volterra recurrent neural networks," *Neurocomputing*, vol. 138, pp. 157–166, Aug. 2014.

[6] Z. Yi and K. K. Tan, "Dynamic stability conditions for Lotka-Volterra recurrent neural networks with delays," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 66, no. 1, Jul. 2002, Art. no. 011910.

[7] Z. Yi, J. Cheng Lv, and L. Zhang, "Output convergence analysis for a class of delayed recurrent neural networks with time-varying inputs," *IEEE Trans. Syst., Man Cybern. B, Cybern.*, vol. 36, no. 1, pp. 87–95, Feb. 2006.

[8] J. Wang, L. Zhang, Y. Chen, and Z. Yi, "A new delay connection for long short-term memory networks," *Int. J. Neural Syst.*, vol. 28, no. 6, Aug. 2018, Art. no. 1750061.

[9] L. Zhang, Z. Yi, and S.-I. Amari, "Theoretical study of oscillator neurons in recurrent neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5242–5248, Nov. 2018.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.

[11] K. Cho *et al.*, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.
[12] R. J. Williams and D. Zipser, *Gradient-Based Learning Algorithms for Recurrent Networks and Their Computational Complexity*. Hillsdale, MI, USA: NewPsychology press, 1995, pp. 433–486.
[13] Z. Yi and K. K. Tan, *Convergence Analysis of Recurrent Neural Networks*, vol. 13. Boston, MA, USA: Kluwer, 2004.
[14] Z. Yi, L. Zhang, J. Yu, and K. Kiong Tan, "Permitted and forbidden sets in discrete-time linear threshold recurrent neural networks," *IEEE Trans. Neural Netw.*, vol. 20, no. 6, pp. 952–963, Jun. 2009.
[15] Z. Yi, M. Ye, J. C. Lv, and K. K. Tan, "Convergence analysis of a deterministic discrete time system of Oja's PCA learning algorithm," *IEEE Trans. Neural Netw.*, vol. 16, no. 6, pp. 1318–1328, Nov. 2005.
[16] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Comput.*, vol. 12, no. 10, pp. 2451–2471, Oct. 2000.
[17] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "Lstm: A search space odyssey," *IEEE Trans. Neural Netw.*, vol. 28, no. 10, pp. 2222–2232, 2016.
[18] R. Józefowicz, W. Zaremba, and I. Sutskever, "An empirical exploration of recurrent network architectures," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 2342–2350.
[19] F. A. Gers and J. Schmidhuber, "Recurrent nets that time and count," in *Proc. IEEE-INNS-ENNS Int. Joint Conf. Neural Netw. Neural Comput., New Challenges Perspect. New Millennium (IJCNN)*, Jul. 2000, pp. 189–194.
[20] J. Wang, L. Zhang, Q. Guo, and Z. Yi, "Recurrent neural networks with auxiliary memory units," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1652–1661, May 2018.
[21] N. Kalchbrenner, I. Danihelka, and A. Graves, "Grid long short-term memory," *CoRR*, vol. abs/1507.01526, pp. 1–15, Jul. 2015.
[22] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training very deep networks," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NIPS)*, 2015, pp. 2377–2385.
[23] J. G. Zilly, R. K. Srivastava, J. Koutník, and J. Schmidhuber, "Recurrent highway networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*. PMLR, 2017, pp. 4189–4198.
[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[25] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.
[26] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 1310–1318.
[27] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 1120–1128.
[28] M. Henaff, A. Szlam, and Y. LeCun, "Recurrent orthogonal networks and long-memory tasks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 2034–2042.
[29] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional LSTM and other neural network architectures," *Neural Netw.*, vol. 18, nos. 5–6, pp. 602–610, Jul. 2005.
[30] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," *CoRR*, vol. abs/1409.2329, pp. 1–8, Sep. 2014.
[31] S. Wisdom, T. Powers, J. R. Hershey, J. L. Roux, and L. E. Atlas, "Full-capacity unitary recurrent neural networks," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NIPS)*, 2016, pp. 4880–4888.
[32] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 1422–1432.
[33] Y. Liu and M. Lapata, "Learning structured text representations," *Trans. Assoc. Comput. Linguistics*, vol. 6, pp. 63–75, Dec. 2018.
[34] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2016, pp. 1480–1489.
[35] Q. V. Le, N. Jaitly, and G. E. Hinton, "A simple way to initialize recurrent networks of rectified linear units," *CoRR*, vol. abs/1504.00941, pp. 1–9, Apr. 2015.
[36] C. Laurent, G. Pereyra, P. Brakel, Y. Zhang, and Y. Bengio, "Batch normalized recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Mar. 2016, pp. 2657–2661.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:14:55 UTC from IEEE Xplore. Restrictions apply.